

Final Year Project Documentation: EHR Web Application for Hypertensive Patients

Title:

Development of a Web-Based Electronic Health Records (EHR) Application for Hypertensive Patient Management Using Laravel Inertia

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Background of Study
5. System Requirements
6. Project Scope
7. System Architecture
8. Technologies Used
9. Laravel Inertia Explained
10. System Design & Diagrams
11. Database Design
12. Authentication System
13. Frontend Flow (React + Inertia)
14. Patient Use Flow
15. Doctor Use Flow

- 16. Admin Dashboard
 - 17. Security & Validation
 - 18. Testing Strategy
 - 19. Challenges and Solutions
 - 20. Conclusion
-

1. Introduction

Hypertension, or high blood pressure, is a leading risk factor for heart disease and stroke. Managing this condition requires regular health monitoring, lifestyle management, and frequent consultations with doctors. This project introduces a digital transformation of the traditional health record system using a modern full-stack web approach combining Laravel with Inertia.js and React, enabling hypertensive patients to engage with their health proactively.

2. Problem Statement

Traditional health records and communication between patients and doctors often rely on paper-based systems or scattered data across different platforms. For hypertensive patients, this inefficiency can delay critical care. There is an evident need for a unified digital system that simplifies access to health records, facilitates doctor-patient communication, and supports real-time health tracking.

3. Objectives

- **Build a secure role-based web application using Laravel with Inertia.js and React:**
The application needs to distinguish between different types of users (patients, doctors, and admins). Laravel's built-in authorization and middleware allow the system to protect routes and manage permissions based on roles. Inertia.js helps in managing the routing from the Laravel backend while rendering React components, providing a smooth SPA-like experience.
- **Create an interactive dashboard for patients to log blood pressure readings:**
Patients should be able to input their systolic and diastolic blood pressure readings, which are stored in the database and visualized using charts. The dashboard dynamically reflects updates, giving patients a clear view of their health trends.

- **Enable doctors to view and monitor patient health data and send messages:** Doctors will have access to patient-submitted data in a clean and structured dashboard. They can review historical readings and communicate with their patients through a secure messaging feature integrated in the platform.
- **Implement appointment scheduling, medical history management, and real-time chart visualization:** Patients can schedule appointments by selecting doctors and available time slots. Historical appointments and health logs are stored and accessible for both patients and doctors. Real-time visualization tools (like Chart.js) enhance the monitoring experience.
- **Provide admins with system-wide control of users and data:** Admin users are responsible for overall system oversight, including managing user accounts, resetting passwords, viewing logs, and maintaining access control. This ensures the application remains secure, well-organized, and monitored.
- Create an interactive dashboard for patients to log blood pressure readings
- Enable doctors to view and monitor patient health data and send messages
- Implement appointment scheduling, medical history management, and real-time chart visualization
- Provide admins with system-wide control of users and data

4. Background of Study

As healthcare transitions into the digital age, chronic conditions such as hypertension demand scalable solutions. Paper-based systems often lead to fragmented care, errors, and inefficiencies. By leveraging Laravel's backend capabilities with Inertia.js's ability to bridge server-driven apps with rich frontend experiences, this EHR system provides a responsive and structured approach to chronic care management.

5. System Requirements

Software:

- Laravel 12
- Inertia.js
- React 18

- MySQL 8+
- Tailwind CSS
- GSAP / Framer Motion

Hardware:

- A server (local or production) capable of running PHP 8+
- Browser with modern JavaScript support

6. Project Scope

The scope of this project outlines what the system will address, what it will provide to its users, and the extent to which the features are implemented:

- **Patient Health Monitoring:** Patients will have the ability to log their blood pressure readings over time. These records are displayed visually through charts to help both the patient and doctor understand trends in blood pressure. The application ensures real-time data saving, feedback after input, and visual representation for easier interpretation.
- **Appointment Scheduling:** A calendar-based interface allows patients to schedule medical appointments. Doctors can accept or reschedule these requests. Notifications and reminders are integrated into the scheduling system to ensure timely consultations.
- **Doctor-Patient Chat:** A secure internal messaging feature is implemented, allowing patients and doctors to communicate. This reduces the need for external communication apps and ensures that communication is confidential and focused on patient health.
- **Role-Based Access Control:** Each user has a designated role—patient, doctor, or admin—with specific access rights. Patients can only access their own records and request services. Doctors can view patient data assigned to them. Admins can manage all users and oversee the system.
- **Admin Controls:** Admins are empowered to create new user accounts, assign roles, reset passwords, manage appointments system-wide, and view audit logs for system changes. This maintains the integrity and smooth operation of the EHR platform.

Excluded from the project:

- **Mobile App Version:** While a mobile-friendly design is considered using responsive CSS, no dedicated mobile application (Android/iOS) is built in this phase.
- **AI-Based Diagnosis:** The platform does not make automatic health assessments or diagnoses based on the data. All medical interpretations are left to the doctor. Integration of AI models is a potential future extension.
- Mobile app version (future enhancement)
- AI-based diagnosis (out of current scope)

7. System Architecture

Client-Server-Database model:

[React + Inertia Frontend] <--> [Laravel Controllers & Routes] <--> [MySQL Database]

Inertia allows seamless interaction where pages are loaded via Laravel routes but behave like a SPA (Single Page Application) with no full-page reloads.

8. Technologies Used

This project integrates several modern technologies, each chosen for its performance, scalability, and ecosystem support. Here's a breakdown of each technology and its role in the system:

Technology	Purpose
Laravel	Acts as the server-side framework. Laravel handles routing, business logic, middleware, validation, and database interaction using Eloquent ORM. It's also used for handling authentication through Sanctum.
Inertia.js	Connects Laravel with the frontend React components. It allows React components to be rendered from Laravel routes without a traditional REST API. This reduces complexity and ensures the system behaves like a Single Page Application (SPA).
React	React is used to build interactive and reusable UI components on the frontend. With Inertia, each page is a component and can manage state, handle props, and interact dynamically with the user.

MySQL	Serves as the primary data storage engine. It stores patient information, appointment records, blood pressure logs, and messages in a structured relational format.
Tailwind CSS	A utility-first CSS framework used to rapidly build modern, responsive user interfaces. It ensures a consistent and mobile-friendly layout.
Sanctum	Laravel Sanctum provides lightweight authentication for SPAs and secures the API using tokens. It simplifies user login, session management, and route protection.
GSAP / Framer Motion	These libraries are used to animate UI elements, providing smooth transitions, page reveals, and interactive effects that improve user engagement and the overall experience.
Heroicons	Heroicons offers a consistent set of clean and minimalistic SVG icons used across the application to enhance UI intuitiveness and accessibility.

Each of these technologies contributes to creating a maintainable, responsive, and secure system designed specifically for managing hypertensive patient records and communication.

Purpose

Laravel	Backend framework & API logic
Inertia.js	Server-driven single-page routing
React	UI components
MySQL	Relational database
Tailwind CSS	Frontend styling
Sanctum	Token-based authentication
GSAP/Framer	Smooth page transitions and animations
Heroicons	Interface icons

9. Laravel Inertia Explained

Inertia.js is a modern approach that allows you to build single-page applications (SPAs) using traditional server-side routing. It acts as a glue between Laravel and a frontend framework like React, allowing you to create a reactive user interface without needing to build and consume a separate REST or GraphQL API.

How It Works:

- Laravel handles the backend logic and routes as usual.
- Instead of returning a Blade view, Laravel returns an Inertia response that loads a JavaScript component (e.g., a React page).
- This page receives server-side data as props and renders dynamically in the browser.

Key Concepts:

- `Inertia::render('ComponentName', [data])` is used to send a React component and data from the controller.
- Pages are structured in `/resources/js/Pages`, similar to traditional views.
- Layouts are shared across pages, enabling consistent headers, footers, and sidebars.
- React handles reactivity, state, and component rendering, while Laravel controls routing and logic.

Benefits:

- **SPA-like Experience Without API Hassle:** Developers don't need to create separate endpoints for each view or data request.
- **Shared Middleware and Auth:** Laravel middleware can protect pages, and Inertia shares auth and session data without duplication.
- **Faster Page Transitions:** Inertia replaces only the page content on route change, not the whole page, improving performance.
- **Simplified Development:** Full-stack features (like form validation, redirects, flash messages) can still be used just like in a traditional Laravel application.

Example Workflow:

- A user logs in via `/login`, handled by Laravel.
- Laravel authenticates and then responds with `Inertia::render('Dashboard', ['user' => Auth::user()])`.
- React renders the `Dashboard` page and receives `user` as a prop.

Why It's Ideal for This Project:

- It allows rapid prototyping while retaining Laravel's backend power.
- React handles interactive components like blood pressure charts, appointment calendars, and messaging UI.
- There's no need to maintain a separate API infrastructure, reducing development time and complexity.

By using Laravel with Inertia and React, the system delivers a highly interactive, modern experience while keeping the robust features of Laravel intact. between Laravel backend and React frontend. It allows you to:

- Use Laravel routes as you would in traditional apps
- Render React components without API overhead
- Manage auth, middleware, and sessions seamlessly

Key Inertia Concepts:

- `Inertia::render('Component', [data])`: Laravel passes props directly to React components.
- React pages exist in `/resources/js/Pages`
- Layouts can be shared across pages just like with Blade.

Advantages:

- No REST API needed for most features

- React state + Laravel backend integration
- Fast SPA-like experience

10. System Design & Diagrams

System design involves understanding how users interact with the platform, how data flows between components, and what key controllers and modules work together to deliver functionality. The EHR system is composed of three primary user roles—Patient, Doctor, and Admin—each with their specific workflows and access.

Use Case Diagram Explanation:

This diagram represents how each user role interacts with the system:

- **Patient:** Registers or logs in, then logs blood pressure readings daily. Can view trends on their dashboard, send messages to doctors, and book medical appointments.
- **Doctor:** Logs in, views a list of assigned patients, checks historical blood pressure logs, replies to patient messages, and manages appointment confirmations.
- **Admin:** Has full control to manage users (create, delete, reset passwords), oversee appointments, and generate system reports.

Component Diagram Explanation:

Each module of the system is organized as a Laravel controller, performing a specific business logic:

- **AuthController:** Handles login, logout, and registration processes, utilizing Sanctum for token authentication.
- **PatientDashboardController:** Manages all patient-side functionality, including blood pressure logging, chart data retrieval, and appointment views.
- **DoctorDashboardController:** Provides doctors with access to patient records, messages, and appointment confirmations.
- **AdminController:** Offers admin tools like user management, access control, and system metrics.

- **BloodPressureController**: Responsible for storing, validating, and displaying blood pressure readings. Supports graph rendering using libraries like Chart.js.
- **AppointmentController**: Handles the logic for creating, modifying, confirming, and deleting patient appointments.
- **MessageController**: Facilitates real-time messaging between patients and doctors using Laravel events or polling.

These modules work in sync through Laravel's MVC architecture and are dynamically rendered using React via Inertia.

Component Diagram:

- **AuthController**
- **PatientDashboardController**
- **DoctorDashboardController**
- **AdminController**
- **BloodPressureController**
- **AppointmentController**
- **MessageController**

11. Database Design

The database design defines the structure of how information is stored and related in the system. Since this is an EHR for hypertensive patients, careful attention is given to how user roles, health records, and communication data are structured and interlinked.

Core Tables and Entities:

- **Users Table**: This stores login credentials and personal details of all users. A **role** field (enum or string) distinguishes between patients, doctors, and admins. Other fields include **name**, **email**, **password**, and timestamps.
- **BloodPressures Table**: Contains individual blood pressure readings submitted by patients. Fields include **user_id**, **systolic**, **diastolic**, **reading_date**, and optional notes.

A foreign key links it to the user (patient).

- **Appointments Table:** Records appointment requests between patients and doctors. Fields include `patient_id`, `doctor_id`, `appointment_date`, `status` (pending/confirmed/cancelled), and optional notes.
- **Messages Table:** Used for internal communication between patients and doctors. Fields include `sender_id`, `receiver_id`, `message_body`, `read_status`, and `sent_at`. This supports secure asynchronous messaging.

Relationships Between Entities:

- A **User (patient)** can have **many BloodPressure** records → One-to-many.
- A **Doctor (user)** can have **many Appointments** assigned to them → One-to-many.
- A **Message** is sent from one **User** to another **User** → Many-to-many (modeled through `sender_id` and `receiver_id` on a single table).

Entity Relationship Overview:

Users

- ├─ hasMany → BloodPressures
- ├─ hasMany → Messages (sent & received)
- └─ hasMany → Appointments (as patient or doctor)

Appointments

- ├─ belongsTo → Patient (`user_id`)
- └─ belongsTo → Doctor (`doctor_id`)

BloodPressures

- └─ belongsTo → Patient (`user_id`)

Messages

- ├─ belongsTo → Sender (`user_id`)
- └─ belongsTo → Receiver (`user_id`)

This relational model ensures data integrity and enables efficient querying for dashboards, charts, and reporting.

Relationships:

- User has many BloodPressures

- Doctor has many Appointments
- Messages: user-to-user with timestamps

12. Authentication System

The authentication system is implemented using **Laravel Sanctum**, which is well-suited for SPAs (Single Page Applications) that utilize token-based security mechanisms.

How Sanctum Works:

Sanctum uses Laravel's cookie-based session authentication combined with CSRF protection, which means the user doesn't need to manage a token manually for every request in most cases.

Login Flow:

1. A user accesses the login route (`/login`).
2. The login form is a React component rendered through Inertia.
3. Upon submission, the form makes a **POST** request to the Laravel backend with the user's credentials.
4. Laravel verifies the credentials and, if valid, sets a session cookie/token for the user.
5. The authenticated user is redirected to their respective dashboard based on their role.

Sanctum Setup Summary:

- Sanctum is installed via Composer: `composer require laravel/sanctum`
- Sanctum's middleware (`EnsureFrontendRequestsAreStateful`) is applied to `api.php` routes or web routes, depending on session usage.
- The `auth:sanctum` guard is used to protect routes that require authenticated access.

Role-Based Middleware:

Roles are applied via custom middleware:

```
Route::middleware(['auth:sanctum', 'role:doctor'])->group(function () {
```

```
Route::get('/doctor/dashboard', [DoctorController::class, 'index']);
});
```

This allows for fine-grained access control so that:

- Patients cannot access doctor/admin pages
- Doctors only see their assigned patients
- Admins can access all areas

Logout:

Logging out is handled by clearing the Sanctum token or session using Laravel's built-in `logout()` method.

```
Auth::guard('web')->logout();
```

Session Expiry and Token Refresh:

If token expiration is required, Laravel's default session lifetime can be configured in `config/session.php`, and tokens can be manually revoked if needed.

This approach ensures a **secure, scalable, and efficient authentication flow** without the complexity of OAuth or third-party identity providers for a student-level project.- Laravel Sanctum for SPA authentication

- Login creates token; stored in cookies/local storage
- Role middleware restricts access:

```
Route::middleware(['role:doctor'])->group(function () { ... });
```

13. Frontend Flow (React + Inertia)

The frontend architecture is structured to behave like a single-page application (SPA) using Inertia.js to render React components dynamically, driven by Laravel's routing system.

Initial Page Load:

When a user visits the application:

1. They are directed to a Laravel route, such as `/login`, which returns an Inertia response.
2. Inertia renders the appropriate React component (e.g., `LoginPage`) from `/resources/js/Pages`.
3. Any required props (such as flash messages or errors) are passed from Laravel to the component.

Authentication Flow:

- The user enters login credentials in the React form component.
- Using `Inertia.post('/login', credentials)`, the form data is sent to Laravel.
- Upon validation, Laravel authenticates the user and redirects them via Inertia to the appropriate dashboard based on their role.

Dashboard Rendering:

- The dashboard component (`Dashboard.jsx`) is loaded with user-specific props (e.g., recent BP readings, upcoming appointments).
- Charts (using `Chart.js`) visualize blood pressure history.
- Navigation uses Inertia's `Link` component, which updates the URL and content without full-page reloads.

Data Submission Example:

When a patient submits a blood pressure reading:

- The React form uses `Inertia.post('/blood-pressures', formData)`.
- Laravel validates and stores the reading.
- Upon success, the component receives updated props or a redirect via `Inertia.visit()` to refresh the data view.

Reusable Components:

- Components such as `Navbar`, `Sidebar`, and `FlashMessage` are shared across pages using a `Layout` wrapper.
- Layouts are defined using Inertia's shared layout pattern, allowing consistency in design and state across different views.

SPA-like Transitions:

- GSAP and Framer Motion are used to animate component transitions, loading screens, and modal windows.
- These animations improve user feedback during navigation and make the application more intuitive.

Global State:

- Shared state (e.g., logged-in user info, alerts) is injected globally using Inertia's shared props mechanism.

```
InertiaApp.sharedProps = {
  user: authUser,
  flash: sessionMessages
};
```

This architecture ensures seamless integration between Laravel logic and a modern JavaScript-driven UI, allowing for high interactivity while maintaining server-side reliability.

1. User visits `/login` → React component rendered
2. Laravel validates → returns `Inertia::render('Dashboard', [...])`
3. Dashboard component loads data via props
4. Interactions (e.g., log BP) send `POST` via `Inertia.post('/blood-pressures')`
5. Page updated via client-side routing

14. Patient Use Flow

The patient experience is central to this EHR system. From login to data input and communication, the flow is designed to be seamless, responsive, and informative.

1. Registration and Login:

- New users can register through a simple form that captures personal details like name, email, and password.
- Upon submission, Laravel validates the request and creates the user with a **role** of **patient**.
- Login uses Sanctum to authenticate and redirect to the patient's dashboard.

2. Dashboard Overview:

- The patient's dashboard provides a summarized view of recent health metrics and upcoming appointments.
- It displays blood pressure trends using a Chart.js graph with labels for dates and systolic/diastolic values.
- It uses Inertia to pull fresh data whenever the user logs in or submits new records.

3. Logging Blood Pressure Readings:

- A form allows patients to input their systolic and diastolic pressure along with an optional note.
- On submission (**Inertia.post**), Laravel validates and stores the data.
- If successful, the chart on the dashboard refreshes to show the updated trend line.

4. Appointment Scheduling:

- A calendar component allows patients to choose a doctor, preferred date, and time slot.
- Appointments are stored as **pending** in the database until a doctor confirms or reschedules.

- Notifications or alerts are sent back as flash messages (e.g., "Appointment submitted").

5. Messaging and Feedback:

- Patients can view doctor responses or send health updates via a secure chat interface.
- The chat system uses a simple message thread structure to maintain conversation history.
- Messages include timestamps and read status (e.g., seen, unread).

6. Profile and Logout:

- Patients can update basic profile information.
- Logout is a simple call to Laravel's logout route, which clears the session and returns the user to the login page.

This patient-centered flow ensures users can easily monitor their condition, communicate with healthcare professionals, and manage appointments without leaving the platform.-
Registers/Login

- Views dashboard with charts (chart.js)
- Logs new BP
- Schedules doctor appointments
- Receives responses/messages

15. Doctor Use Flow

Doctors play a critical role in this EHR system as the recipients and interpreters of patient data. Their workflow is designed to be efficient, with streamlined access to patient health logs and communication features.

1. Login and Dashboard:

- Doctors authenticate using the same login interface as other users.

- Upon successful login, they are redirected to the doctor dashboard, filtered by their role.
- The dashboard provides an overview of their assigned patients, recent appointments, and unread messages.

2. Viewing Assigned Patients:

- Doctors can access a list of all patients assigned to them, with search and filter options (e.g., by name or health status).
- Selecting a patient opens their health record page, displaying recent and historical blood pressure logs.
- Data is visualized in charts with filtering options (last 7 days, 30 days, custom range).

3. Reviewing Blood Pressure History:

- Detailed views show all readings logged by the patient.
- Doctors can see trends over time, add annotations, or flag readings for follow-up.
- Blood pressure trends are rendered using Chart.js with color indicators for out-of-range values.

4. Appointment Management:

- The appointments module lists all pending, upcoming, and completed appointments.
- Doctors can confirm or reschedule appointments, triggering updates in the patient dashboard.
- Appointments can be synced to external calendars (Google/Outlook) if integrated.

5. Doctor-Patient Messaging:

- A chat interface allows doctors to send secure messages to patients.
- Threads are organized by patient and sorted by latest activity.

- Doctors can give advice, request clarification on symptoms, or follow up on previous consultations.

6. Profile and Logout:

- Doctors can update their availability or personal contact info.
- Logout ends the session securely and redirects them to the login screen.

This flow ensures that doctors are well-equipped to monitor, analyze, and respond to patient needs efficiently within the digital platform.- Login → Dashboard with assigned patients

- Views logs via `/patients/{id}/bp`
- Confirms/Declines appointments
- Sends messages and notes to patients

16. Admin Dashboard

The admin panel provides full oversight and management capabilities across the application. It is designed for ease of use and accountability, enabling administrators to monitor system-wide health, control user access, and enforce data integrity.

1. Access and Role Restriction:

- Admins log in via the same portal but are redirected to a dedicated admin dashboard.
- Role-based middleware ensures only users with `role = admin` access this section.

2. User Management:

- A searchable table lists all users, categorized by role (patient, doctor, admin).
- Admins can:
 - Create or onboard new users (e.g., assigning doctor roles)
 - Update user profiles and passwords

- Suspend or delete user accounts
- Role management allows reassigning user roles dynamically.

3. Appointment Oversight:

- Admins can view all scheduled, pending, or completed appointments across the system.
- This visibility allows them to monitor bottlenecks in consultation flow and resolve scheduling conflicts.

4. Logs and Metrics:

- A system activity feed tracks key actions like login events, profile updates, or blood pressure entries.
- Summary metrics include:
 - Total active users by role
 - Number of BP readings recorded this week
 - Upcoming appointments
 - Message traffic statistics

5. Audit and Monitoring:

- Optional audit trail functionality can log sensitive actions (like data deletions or role changes).
- Helps ensure accountability and support in compliance-driven environments.

6. Notifications and Messaging:

- Admins can broadcast messages (e.g., system maintenance alerts) to all users or role-specific groups.
- Can review unread or failed message delivery statuses.

This admin dashboard enables a centralized, secure command center to manage the EHR platform efficiently while maintaining transparency and compliance across user activities.-
List all users by role

- Reset user passwords
- Delete or suspend accounts
- View system metrics and activity logs

17. Security & Validation

Security and input validation are critical in healthcare applications due to the sensitivity of patient data. This EHR system uses multiple layers of built-in and custom protection mechanisms.

1. Form Request Validation:

- Laravel's Form Request classes are used to encapsulate validation logic.
- Fields such as `email`, `password`, `systolic`, and `diastolic` are validated using Laravel's `rules()` method before any database operation.
- Example:

```
public function rules()
{
    return [
        'email' => 'required|email',
        'password' => 'required|min:8',
        'systolic' => 'required|numeric|min:70|max:250',
        'diastolic' => 'required|numeric|min:40|max:150'
    ];
}
```

2. CSRF Protection:

- Laravel includes built-in CSRF protection via middleware for all web routes.
- All forms include the `@csrf` directive or hidden input tokens.

- Inertia's requests automatically include CSRF tokens via Axios headers, maintaining full protection.

3. XSS and HTML Escaping:

- All user input displayed in views is automatically escaped by React and Laravel.
- Messages and notes are sanitized on entry and stored safely in the database.
- Use of third-party libraries like DOMPurify (optional) further enhances HTML content safety if needed.

4. Encrypted Tokens (Sanctum):

- Laravel Sanctum uses signed/encrypted cookies to authenticate users.
- These tokens are stored securely and not exposed to JavaScript unless explicitly needed.
- Prevents token hijacking and supports revoking on logout.

5. Role-Based Access Middleware:

- Middleware ensures that each user only accesses what is allowed.
- Even if a route is guessed (e.g., `/admin/users`), users are redirected or denied access if they don't have the correct role.

6. Password Hashing:

- All passwords are hashed using `bcrypt` before storage.
- Laravel's Auth system uses `Hash::make()` internally, and never stores plain-text credentials.

7. Optional Audit Trails:

- For admin actions like account deletion or role reassignment, Laravel's built-in logging or packages like Spatie Activity Log can be used.

- Logs are timestamped, user-tagged, and stored in a secure table for traceability.

By combining Laravel's defaults with deliberate best practices in validation, session handling, and access control, the system offers strong protection for sensitive healthcare data while preserving usability for all users.- Form request validation

- XSS/CSRF protected via Laravel defaults
- Encrypted tokens
- Logged actions (audit trail optional)

18. Testing Strategy

A well-defined testing strategy ensures the reliability and functionality of the application across all user roles and critical workflows. This project incorporates a blend of manual, automated, and exploratory testing techniques to validate each component of the system.

1. Manual Testing:

- Each route and UI component is manually tested using the browser's developer tools to inspect props, responses, and interactions.
- Cross-browser testing (Chrome, Firefox, Edge) ensures visual consistency and proper animation rendering.
- Mobile responsiveness is verified using browser tools and actual devices.

2. Postman API Testing:

- Used for testing Laravel routes independently of the frontend.
- API endpoints such as `/login`, `/logout`, `/blood-pressure`, and `/appointments` are tested for expected input/output behavior.
- Edge cases (invalid data, unauthorized access) are simulated to confirm correct HTTP status codes and error handling.

3. PHPUnit Backend Testing:

- Laravel's built-in PHPUnit framework is used to write unit tests and feature tests.
- Example unit test: Testing that a **BloodPressure** record is only saved with valid systolic/diastolic values.
- Feature tests simulate user flows, such as login, role redirection, and form validation.

4. Frontend Testing (Optional):

- React component logic is optionally tested using libraries like Jest or React Testing Library (if implemented).
- Validates state updates, prop passing, and form behavior.

5. Data Validation Tests:

- Form Request classes are tested to confirm that invalid data (e.g., non-numeric BP values) trigger validation errors.
- Client-side error messages are checked to ensure users receive meaningful feedback.

6. User Role Access Testing:

- Tests confirm that each route is accessible only to the correct role (patient, doctor, admin).
- Middleware and redirect logic are verified through automated feature tests.

7. Inertia Navigation Checks:

- Navigating between pages is tested to ensure transitions are smooth, props are passed correctly, and no full-page reloads occur.
- GSAP/Framer Motion animations are checked to ensure they trigger at correct points in the flow.

This multi-layered testing approach ensures stability, data integrity, and a smooth user experience for patients, doctors, and administrators alike. - Manual testing via browser/devtools

- Postman for API testing (where needed)
- PHPUnit for backend logic
- Browser-based test cases for pages and transitions

19. Challenges and Solutions

Every software project presents unique challenges, especially in health tech where security, performance, and UX are crucial. Here are the main challenges encountered during the development of this EHR system and the corresponding solutions applied:

1. Routing Confusion with Inertia and Laravel:

- **Problem:** Initially, managing page transitions using Laravel's route definitions alongside Inertia components created conflicts, especially when determining which component should be rendered and where.
- **Solution:** The solution was to define all logical routes on the Laravel backend and pair them directly with Inertia's `render()` method. Pages were modularized under `/resources/js/Pages`, and each route returned an Inertia-rendered React component. A shared layout structure was introduced to keep navigation and layout consistent across all pages.

2. Role-Based Access Management:

- **Problem:** Without strict control, it was possible for users to manually access pages outside their roles (e.g., a patient accessing admin routes).
- **Solution:** Laravel middleware (`auth:sanctum` and `role`) was applied to each route group. These route groups check user roles before granting access. Unauthorized access leads to redirection with a flash message explaining the restriction.

3. Data Synchronization Between Pages:

- **Problem:** When data was updated (e.g., blood pressure readings), the page sometimes displayed stale props without refreshing.
- **Solution:** Implemented `Inertia.visit()` and `Inertia.reload()` to refresh component props after data mutation. Shared props (like auth data, flash messages) were made globally available via Inertia's shared props configuration to ensure state consistency.

4. Managing API Tokens and Expiry:

- **Problem:** API sessions expired without proper notification, forcing users to refresh or re-login unexpectedly.
- **Solution:** Laravel's session timeout was configured appropriately (`config/session.php`) with reasonable lifespans. A logout check mechanism was added to detect expired sessions and prompt re-authentication smoothly.

5. Cross-Component State Handling:

- **Problem:** Patient readings and appointment updates needed to be reflected in multiple parts of the UI, sometimes leading to inconsistent data.
- **Solution:** React's component hierarchy was carefully managed to lift state where necessary and share updated props via Inertia re-renders. Transient states (like loading or success messages) were stored in local state within the component.

6. Animation and UX Transitions:

- **Problem:** Animation timing conflicts occasionally occurred between Framer Motion and Inertia navigation, causing page flickers.
- **Solution:** Delays and exit animations were properly synchronized using lifecycle hooks in React (e.g., `useEffect`, `useLayoutEffect`) to ensure that animation transitions completed before the DOM was updated.

7. Maintaining Component Reusability:

- **Problem:** Multiple forms (e.g., login, blood pressure input) shared similar validation and UI elements, leading to repetitive code.
- **Solution:** Abstracted these forms into reusable components (e.g., `<InputField />`, `<FormError />`) and passed in props for flexibility. This improved maintainability and code clarity.

By tackling these technical and architectural challenges methodically, the system's structure became more robust, scalable, and easier to maintain. These lessons helped shape a cleaner implementation and better user experience across the platform.- **Routing Confusion:** Solved by mapping Laravel routes to Inertia correctly

- **Data Sync:** Used `Inertia.shared()` for global props
- **Token Expiry:** Implemented token refresh mechanism

20. Conclusion

This final year project has successfully delivered a modern, web-based Electronic Health Records (EHR) system specifically tailored for hypertensive patients. The combination of Laravel, Inertia.js, and React offers a powerful and interactive platform that addresses critical challenges in patient data management and remote healthcare monitoring.

Key Achievements:

- The implementation of role-based access ensures that each user—patient, doctor, or admin—has a tailored experience with appropriate permissions.
- Patients can conveniently record and visualize their blood pressure readings using intuitive dashboards and charts.
- Doctors can analyze patient data in real-time, manage appointments, and communicate effectively within a secure, structured interface.
- Admins have the tools to oversee the system, manage users, and ensure compliance and data integrity.

Technical Highlights:

- Inertia.js serves as a bridge between Laravel and React, eliminating the need for a separate API layer and enabling SPA-like transitions with server-side routing.
- Laravel Sanctum secures user sessions and facilitates smooth authentication flows for SPAs.
- React components, enhanced by GSAP and Framer Motion, deliver a modern, animated, and responsive user interface.

Impact and Future Scope:

This EHR system demonstrates how full-stack web technologies can be used to solve real-world health challenges. By streamlining communication and enabling real-time health monitoring, it empowers patients to actively manage their condition and allows healthcare providers to deliver timely, data-informed care.

In the future, this system can be extended with features such as:

- Integration with wearable devices for automated health data input
- AI-based health analytics and alert systems
- Dedicated mobile apps for Android and iOS

Ultimately, this project exemplifies the potential of technology in healthcare, showcasing a viable, scalable solution that aligns with modern clinical workflows and digital transformation goals in medical practice.